

# **Introdução à Análise Exploratória de Dados**

**Autor:** Eli Kleber Carneiro Barreto  
Elias Guideon Carneiro Barreto  
Emerson Lucas Sacramento Lima

# Agenda

O objetivo dessa apresentação é detalhar a implementação técnica e o cálculo dos seguintes operadores estatísticos:

## Contexto e Classificação dos Operadores

### **1. Contextualização e Objetivos:**

Explicar AED e a tradução de fórmulas matemáticas para pré-processamento de dados

### **2. Operadores de Tendência Central:**

Média, Mediana e Moda.

### **3. Operadores de Dispersão e Posição:**

Variância, Desvio Padrão e Quartis.

### **4. Frequências:**

Absoluta, Relativa e Acumulada.

### **5. Análise Avançada:**

Covariância, Probabilidade Condicional, Itemset e Histograma.

# 1. Contextualização

- A Análise Exploratória de Dados (AED) é a base para identificar padrões e tendências antes da modelagem complexa.
- O projeto surge da necessidade de processar dados sem dependências externas, utilizando **Python nativo** para garantir controle total sobre a lógica.
- O objetivo é preparar os dados para realizar conversões e leituras de forma rápida e precisa, a partir de operadores matemáticos, fazendo-se necessária a adaptação de alguns dados.

# 1.1 Contextualização: Dados Numéricos e Categóricos

- Dentro da AED, um fenômeno natural dos códigos acontece: Diferenças entre tipos de dados recebidos. Muitos dos dados podem vir de diferentes formas, porém, os mais utilizados são palavras e numerais. (String e Int/Float).

```
"priority": [  
    "alta", "media", "alta", "baixa", "alta",  
    "media", "baixa", "alta", "baixa", "alta"  
]
```

Imagem da Tabela "Priority"

```
# Numéricas  
"participants": [120, 80, 150, 40, 200, 90, 60, 180, 55, 160],  
"duration_hours": [2,3,4,2,5,3,2,4,2,3],  
"ticket_price": [50,30,70,20,80,30,25,75,20,65],  
"rating": [4.5,4.0,4.8,3.9,4.9,4.2,4.0,4.7,3.8,4.6]
```

Imagem das Tabelas Numéricas

# 1.1 Contextualização: Dados Numéricos e Categóricos

- Um dos desafios é a própria adaptação dos dados, pois como lidar com operadores matemáticos usando dados em formato “strings”? A resposta são códigos que fazem com que os dados repetidos em strings, em uma tabela específica, assumam um valor numeral, assim tornando possível a operação matemática.

```
"priority": [  
    "alta", "media", "alta", "baixa", "alta",  
    "media", "baixa", "alta", "baixa", "alta"  
]
```

Imagem da Tabela “Priority”

```
# Criando valores baseados nos dados da coluna "priority"  
if column == "priority":  
    ordem = {"baixa": 0, "media": 1, "alta": 2}
```

Exemplo da “transformação” de string em numeral

## 2. Operadores de Tendência Central

- Operador Média (Mean):

**Cálculo Matemático:** Representa o valor central obtido pela soma de todos os elementos dividida pela quantidade total.

### Aplicação no Código:

- **Acumulação:** O código percorre a lista através de um laço `for`, armazenando a `soma_total` e o contador `quantidade_elementos`.
- **Cálculo Final:** Retorna a divisão da soma pelo contador.
- **Validação:** Verifica se o primeiro item é numérico antes de iniciar a iteração para evitar erros de tipo.

Exemplo do Cálculo:

$$\text{Fórmula: } \bar{x} = \frac{\sum_{i=1}^n x_i}{n}$$

$$\text{Cálculo: } \frac{3+4+5}{3} = \frac{12}{3} = 4$$

## 2. Operadores de Tendência Central

- Operador Média (Mean) – Implementação

```
def mean(self, column):  
  
    # Pegando os dados dentro do dataset  
    dados = self.dataset[column]  
  
    # Verificando se os dados são números  
    if not isinstance(dados[0], (int, float)):  
        raise ValueError(f"ERRO: A coluna '{column}' não é numérica.")  
  
    soma_total = 0  
    quantidade_elementos = 0  
  
    for valor in dados:  
        soma_total += valor  
        quantidade_elementos += 1  
  
    media = soma_total / quantidade_elementos  
  
    return media
```

## 2. Operadores de Tendência Central

- Operador Mediana (Median):
- **Cálculo Matemático:** É o valor que separa a metade superior da inferior. Exige ordenação prévia. Caso  $n$  seja par, calcula-se a média dos dois valores centrais.

### Aplicação no Código:

- **Ordenação:** Utiliza o método nativo `sorted()`.
- **Lógica de Prioridade:** Implementa uma função `lambda` para tratar a coluna `priority`, convertendo strings ("baixa", "media", "alta") em pesos numéricos (0, 1, 2) para permitir a ordenação lógica.
- **Posicionamento:** Usa o operador de divisão inteira (`//`) para localizar o índice central.

Se  $n$  é ímpar:  $\text{Mediana} = x_{(\frac{n+1}{2})}$

Se  $n$  é par:  $\text{Mediana} = \frac{x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}}{2}$



## 2. Operadores de Tendência Central

- Operador Mediana (Median):

**Exemplo:**

**Fórmula:** Valor central de um conjunto ordenado. Se  $n$  é ímpar, a posição é  $(n+1) / 2$

**Cálculo:** {3,4,5}. A posição central é o segundo elemento: **4**

## 2. Operadores de Tendência Central

- Operador Mediana (Median) - Implementação

```
def median(self, column): 4 usages EliBarreto1 +1

    dados = self.dataset[column]
    # Criando valores baseados nos dados da coluna "priority"
    if column == "priority":
        ordem = {"baixa": 0, "media": 1, "alta": 2}

        # Ordenando os valores e validando se está corretamente numérica e com ordem
        dados_ordenados = sorted(dados, key=lambda x: ordem[x])
    else:
        try:
            dados_ordenados = sorted(dados)
        except TypeError:
            raise ValueError(f"ERRO: A coluna '{column}' não é numérica e não tem ordem definida.")

    # Fórmula da Mediana
    n = len(dados_ordenados)
    meio = n // 2

    if n % 2 != 0:
        return dados_ordenados[meio]
    else:
        if isinstance(dados_ordenados[meio], (int, float)):
            return (dados_ordenados[meio - 1] + dados_ordenados[meio]) / 2
        else:
            return dados_ordenados[meio - 1]
```

## 2. Operadores de Tendência Central



- Operador Moda (Mode):
- **Cálculo Matemático:** Identifica os valores que apresentam a maior frequência absoluta dentro do conjunto de dados.

Exemplo:  
Conjunto {3,3,4,4,4,5,5}  
Moda = 4

### Aplicação no Código:

- **Mapeamento de Frequência:** O algoritmo utiliza um dicionário (**contagem**) para armazenar cada item como chave e a quantidade de ocorrências como valor.
- **Identificação do Máximo:** Utiliza a função **max()** sobre os valores do dicionário para encontrar a maior frequência registrada.
- **Extração de Modas:** Percorre o dicionário e seleciona todas as chaves que atingiram o valor máximo, garantindo o suporte a distribuições bimodal ou multimodal.

## 2. Operadores de Tendência Central

- Operador Moda(Mode) – Implementação

```
def mode(self, column):  
    dados = self.dataset[column]  
  
    # Criando um dicionário para contar ocorrências  
    contagem = {}  
  
    # Criando fórmula de contagem, os itens serão associados à quantidade de vezes em que aparecem,  
    # assumindo um valor  
  
    for item in dados:  
        if item in contagem:  
            contagem[item] += 1  
        else:  
            contagem[item] = 1  
  
    # Max retorna o maior valor entre os itens contados anteriormente  
    max_frequencia = max(contagem.values())  
  
    # Criando fórmula de Moda, verifica o valor adquirido por cada item e compara com o maior registrado no 'max',  
    # O item que for igual ao 'max', será considerado o contado mais vezes, ou seja, a Moda  
    modas = []  
    for item in contagem:  
        if contagem[item] == max_frequencia:  
            modas.append(item)  
  
    # Retorna a Moda  
    return sorted(modas)
```

### 3. Operadores de Dispersão e Posição



- Operador Variância (Variance)

**Cálculo Matemático:** É a média aritmética dos quadrados dos desvios em relação à média.

$$\sigma^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}$$

#### Aplicação no Código:

- **Cálculo de Resíduos:** O código calcula a diferença  $(x_i - \bar{x})$  para cada elemento da coluna.
- **Acumulação:** Utiliza o operador de potência `** 2` dentro de um laço `for` para acumular a soma das diferenças quadradas na variável `soma_varianca`.

### 3. Operadores de Dispersão e Posição



- Operador Variância (Variance)

Exemplo:

**Cálculo:**  $\frac{(3-4)^2 + (4-4)^2 + (5-4)^2}{3} = \frac{1+0+1}{3} = 0,66\dots$

### 3. Operadores de Dispersão e Posição

- Operador Variância (Variance) – Implementação

```
def variance(self, column): 3 usages  👤 Elias Barreto +1

    dados = self.dataset[column]

    # Reuso da media
    media_valores = self.mean(column)

    # Cálculo da soma das diferenças ao quadrado
    soma_varianca = 0
    for valor in dados:
        diferenca = valor - media_valores
        soma_varianca += diferenca ** 2

    # Divisão pelo total de elementos
    return soma_varianca / len(dados)
```

### 3. Operadores de Dispersão e Posição



- Operador Desvio Padrão (Stdev)

**Cálculo Matemático:** Representa a raiz quadrada da variância, trazendo a dispersão de volta à unidade original dos dados.

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

#### Aplicação no Código:

- **Reuso de Operador:** O método chama internamente a função `self.variance(column)`.
- **Raiz Nativa:** Aplica a potência `** 0.5` para extrair a raiz quadrada sem necessitar de bibliotecas externas como a `math`.



### 3. Operadores de Dispersão e Posição



- Operador Desvio Padrão (Stdev)

**Exemplo:**

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

**Fórmula:**  $\sigma = \sqrt{\sigma^2}$

**Cálculo:**  $\sqrt{0,66...} \approx 0,816$

### 3. Operadores de Dispersão e Posição

- Operador Desvio Padrão (Stdev) – Implementação

```
def stdev(self, column): 2 usages  Elias Barreto +1  
  
    # Reuso da variância  
    valor_variancia = self.variance(column)  
  
    # Cálculo da raiz quadrada  
    resultado = valor_variancia ** 0.5  
  
    return resultado
```

### 3. Operadores de Dispersão e Posição



- Operador Quartis (Quartiles)

**Cálculo Matemático:** Divide o conjunto ordenado em quatro partes. O cálculo da posição é dado por  $P_k = \frac{k(n+1)}{4}$  para  $k \in \{1, 2, 3\}$ .

#### Aplicação no Código:

- **Interpolação:** O código define uma função interna `calcular_posicao` que utiliza pesos para interpolar valores quando a posição teórica cai entre dois índices da lista.
- **Ordenação:** Exige que a lista de entrada seja previamente processada pelo método `sorted()`.

### 3. Operadores de Dispersão e Posição

- Operador Quartis (Quartiles)

**Exemplo:**

**Fórmula:** Posição  $P_k = \frac{k(n+1)}{4}$ .

**Cálculo:**  $Q_1 : \frac{1(4)}{4} = 1^{\text{a}} \text{ posição} \rightarrow \mathbf{3}$

$Q_2 : \mathbf{4}$  (Mediana)

$Q_3 : \frac{3(4)}{4} = 3^{\text{a}} \text{ posição} \rightarrow \mathbf{5}$

### 3. Operadores de Dispersão e Posição

- Operador Quartis (Quartiles) – Implementação

```
def quartiles(self, column): 2 usages  Elias Barreto +1

    dados = sorted(self.dataset[column])
    n = len(dados)

    # Reuso da mediana para o Q2
    q2 = self.median(column)

    # Função interna para calcular a posição (Interpolação)
    def calcular_posicao(percentil):  Elias Barreto
        # Localiza o índice (posição teórica)
        indice = percentil * (n + 1)
        idx_baixo = int(indice)
        idx_alto = idx_baixo + 1

        # Ajuste de limites para não sair da lista
        if idx_baixo < 1: return dados[0]
        if idx_baixo >= n: return dados[-1]

        # Calcula o valor entre dois índices (Interpolação)
        peso = indice - idx_baixo
        return dados[idx_baixo - 1] + peso * (dados[idx_alto - 1] - dados[idx_baixo - 1])

    return {
        "Q1": calcular_posicao(0.25),
        "Q2": q2,
        "Q3": calcular_posicao(0.75)
    }
```

## 4. Frequências

- Frequência Absoluta e Relativa

**Frequência Absoluta** é a contagem de um termo em relação ao total de contagem dos termos.

$$f_r = \frac{f_i}{n}$$

**Cálculo Matemático:** A frequência relativa **Fr** é a razão entre a frequência absoluta **Fa** e o total de observações **n**.

### Aplicação no Código:

- **Mapeamento de Itens:** `absolute_frequency` contabiliza cada ocorrência única via dicionário.
- **Normalização:** O método `relative_frequency` itera sobre esse dicionário, dividindo cada valor pelo `len(dataset[column])`.

## 4. Frequências

- Frequência Absoluta

**Fórmula:** Contagem **Fi** de cada item único.

**Cálculo:** {3:1,4:1,5:1}

**Cada item foi contado uma vez.**

- Frequência Relativa

**Fórmula:**  $Fr = Fi / n$

**Cálculo:** Para o valor 3:  $\frac{1}{3} = 0,33... (33,3\%)$ .

O mesmo se repete para **4** e **5**.

O número de contagem é dividido pelo total de **n**.

## 4. Frequências

- Frequência Absoluta – Implementação

```
def absolute_frequency(self, column): 4 usages  Emerson Lucas +1

    dados = self.dataset[column]

    # Dicionário para contar ocorrências
    frequencias = {}

    # Definindo contagem para associar itens a suas frequências
    for item in dados:
        if item in frequencias:
            frequencias[item] += 1
        else:
            frequencias[item] = 1

    return frequencias
```

- Frequência Relativa – Implementação

```
def relative_frequency(self, column): 3 usages  Elias Barreto +1
    # Reaproveitando metodo absolute_frequency utilizado anteriormente
    contagens = self.absolute_frequency(column)

    # Pega o total de linhas na coluna
    total_elementos = len(self.dataset[column])

    # Dicionário para as proporções
    frequencias_relativas = {}

    # Calculando a proporção para cada item
    for item, contagem in contagens.items():
        frequencias_relativas[item] = contagem / total_elementos

    return frequencias_relativas
```



## 4. Frequências

- Frequência Acumulada

**Cálculo Matemático:** É o somatório das frequências das categorias anteriores até a categoria atual  $k$ .

$$F_k = \sum_{i=1}^k f_i$$

### Aplicação no Código:

- **Soma Progressiva:** O código utiliza uma variável `soma_progressiva` que acumula os valores conforme percorre as chaves ordenadas do dicionário.
- **Mapeamento Ordinal:** Se a coluna for `priority`, a acumulação respeita a ordem lógica (baixa -> média -> alta) através de um dicionário de pesos.

## 4. Frequências

- Frequência Acumulada

**Exemplo:**

**Fórmula:**  $F_k = \sum_{i=1}^k f_i$

Para o 3: **1**

Para o 4:  $1 + 1 = \mathbf{2}$

Para o 5:  $2 + 1 = \mathbf{3}$

## 4. Frequências

- Frequência Acumulada – Implementação

```
def cumulative_frequency(self, column, frequency_method='absolute'): 3 usages  Elias Barreto +1
    # Reaproveitando metodo absolute_frequency e relative_frequency utilizado anteriormente
    # Escolhe se usa a contagem numeral ou percentual
    if frequency_method == 'relative':
        frequencia_base = self.relative_frequency(column)
    else:
        frequencia_base = self.absolute_frequency(column)

    # Pega as chaves do dicionário
    categorias = list(frequencia_base.keys())

    # Mesma lógica da Mediana
    if column == "priority":
        ordem_customizada = {"baixa": 0, "media": 1, "alta": 2}
        categorias_ordenadas = sorted(categorias, key=lambda x: ordem_customizada.get(x, default: 0))
    else:
        categorias_ordenadas = sorted(categorias)

    # Lógica da Acumulação
    acumulada = {}
    soma_progressiva = 0

    for item in categorias_ordenadas:
        soma_progressiva += frequencia_base[item]
        acumulada[item] = soma_progressiva

    return acumulada
```

# 5. Análise Avançada

- Operador Covariância (Covariance)

**Cálculo Matemático:** Mede a variabilidade conjunta de duas variáveis aleatórias.

$$cov(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{n}$$

**Aplicação no Código:**

- **Cálculo Bidimensional:** O algoritmo extrai as médias de ambas as colunas simultaneamente.
- **Somatório de Produtos:** Um laço único percorre os índices de 0 a n - 1, multiplicando os desvios de X pelos desvios de Y e acumulando o resultado em `soma_produtos`.

# 5. Análise Avançada

- Operador Covariância (Covariance)

**Exemplo:**

*Nota: Exige dois conjuntos. Usaremos  $X = \{3, 4, 5\}$  e  $Y = \{6, 8, 10\}$ .*

**Fórmula:**  $cov(X, Y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{n}$

**Cálculo:**  $\frac{(3-4)(6-8)+(4-4)(8-8)+(5-4)(10-8)}{3} = \frac{(-1)(-2)+0+(1)(2)}{3} = \frac{4}{3} = 1,33...$

# 5. Análise Avançada

- Operador Covariância (Covariance) – Implementação

```
def covariance(self, column_a, column_b): 2 usages EliBarreto1 +1

    # Capturando os dados das colunas que serão comparadas
    dados_x = self.dataset[column_a]
    dados_y = self.dataset[column_b]
    n = len(dados_x)

    # Raproveitando metodo mean utilizado anteriormente, e extraindo a média em cada coluna
    media_x = self.mean(column_a)
    media_y = self.mean(column_b)

    # Criando a variável para realizar a fórmula da covariância
    soma_produtos = 0

    # Criando a fórmula da covariância (Somatório)
    for i in range(n):
        soma_produtos += (dados_x[i] - media_x) * (dados_y[i] - media_y)
    # Segunda parte da fórmula (divide o somatório pela quantidade de elementos)
    return soma_produtos / n
```

# 5. Análise Avançada

- Probabilidade Condicional

**Cálculo Matemático:** Probabilidade de **A** ocorrer dado **B**. No contexto do código, analisamos a sequência temporal/posicional.

$$P(A|B) = \frac{\text{Contagem de ocorrências de } (B \text{ seguido por } A)}{\text{Contagem total de } B}$$

## Aplicação no Código:

- **Vizinhança Imediata:** O código verifica se `dados[i] == value2` (condição) e, em caso positivo, se `dados[i + 1] == value1` (consequência).
- **Proteção:** Inclui tratamento para evitar divisão por zero caso o evento condicionante nunca ocorra.

## 5. Análise Avançada

- Probabilidade Condicional

**Exemplo:** Probabilidade de vir 4 dado que o anterior foi 3 na sequência [3, 4, 5].

**Fórmula:**  $P(A|B) = \frac{\text{Sequências } (B,A)}{\text{Total de } B}$

**Cálculo:** O valor 3 aparece 1 vez. Em 1 dessas vezes, foi seguido por 4. Logo,  $1/1 = 1,0$  (100%).



# 5. Análise Avançada

- Probabilidade Condicional – Implementação

```
def conditional_probability(self, column, value1, value2): 2 usages Elias Barreto +1

    dados = self.dataset[column]
    n = len(dados)

    # Fórmula:  $P(A|B) = \frac{\text{Contagem de sequências (B, A)}}{\text{Contagem total de B}}$ 

    contagem_b = 0
    contagem_ab = 0

    # O último item não tem ninguém depois dele, então não gera uma sequência
    # Por isso percorre a lista até o penúltimo item
    for i in range(n - 1):
        # Se encontramos o valor condicionante (B)
        if dados[i] == value2:
            contagem_b += 1
            # Verificamos se o próximo item é o valor consequente (A)
            if dados[i + 1] == value1:
                contagem_ab += 1

    # Evita divisão por zero
    if contagem_b == 0:
        return 0.0

    return contagem_ab / contagem_b
```

# 5. Análise Avançada

- Operador Histograma

**Cálculo Matemático:** Agrupamento em intervalos de classe. A largura do bin é dada por

$$L = \frac{X_{max} - X_{min}}{bins}$$

## Aplicação no Código:

- **Dimensionamento de Faixas:** O código calcula a amplitude total e a divide pelo parâmetro `bins` informado.
- **Indexação por Valor:** Para cada dado, o código calcula o índice do intervalo destino através da fórmula `int((valor - min_val) / largura_bin)`, incrementando o contador correspondente.

# 5. Análise Avançada

- Operador Histograma

**Exemplo:** 2 faixas (bins) para {3, 4, 5}.

**Fórmula:**  $Largura = \frac{max-min}{bins} = \frac{5-3}{2} = 1,0$ .

Bin 1 [3, 4): Contém o valor **3**.

Bin 2 [4, 5]: Contém os valores **4 e 5**.

# 5. Análise Avançada

- Operador Histograma - Implementação

```
def histogram(self, column, bins): 2 usages  Elias Barreto +1

    dados = self.dataset[column]

    # Valida e anota os limites
    min_val = min(dados)
    max_val = max(dados)

    # Se todos os valores forem iguais, teremos apenas 1 intervalo
    if min_val == max_val:
        return {(min_val, max_val): len(dados)}

    # Cálculo da largura de cada intervalo (bin)
    amplitude = max_val - min_val
    largura_bin = amplitude / bins

    # Criação dos intervalos e contagem
    histograma = {}
    intervalos = []
```

```
# Logica para os intervalos desejados
for i in range(bins):
    inicio = min_val + (i * largura_bin)
    fim = min_val + ((i + 1) * largura_bin)
    intervalo = (inicio, fim)
    intervalos.append(intervalo)
    histograma[intervalo] = 0

# Distribuí os dados nos intervalos
for valor in dados:
    # Se for o valor máximo, ele entra no último intervalo
    if valor == max_val:
        indice_bin = bins - 1
    else:
        indice_bin = int((valor - min_val) / largura_bin)

    # Proteção contra erros de arredondamento
    if indice_bin >= bins:
        indice_bin = bins - 1

    histograma[intervalos[indice_bin]] += 1

return histograma
```

# 5. Análise Avançada

- Operador Itemset

**Cálculo Matemático:** Representa o conjunto  $U$  de elementos distintos de uma coleção, onde  $x \in U \iff x \in \text{Dataset}$ .

## Aplicação no Código:

- **Unicidade:** Faz uso da estrutura nativa `set()` do Python, que por definição elimina duplicatas e extrai o domínio da coluna analisada.

## 5. Análise Avançada

- Operador Itemset

**Exemplo:**

**Fórmula:**  $U = \{x \mid x \in \text{Lista}\}.$

**Cálculo:** Conjunto de valores únicos: {3, 4, 5}.

## 5. Análise Avançada

- Operador Itemset

```
def itemset(self, column): 2 usages  Emerson Lucas +1  
  
    dados = self.dataset[column]  
  
    # Uso de set para selecionar itens únicos  
    itens_unicos = set(dados)  
  
    return itens_unicos
```

# Limitações Técnicas e de Operação

- **Dependência de Dados Íntegros:** A biblioteca não realiza a imputação de valores ausentes (**None** ou **NaN**), exigindo pré-limpeza.
- **Restrição de Tipagem:** Operadores matemáticos (Variância/Média) interrompem a execução se encontrarem tipos não numéricos, garantindo a precisão, mas exigindo homogeneidade rigorosa.
- **Escalabilidade Nativa:** A implementação via iteração (**for** loops) é ideal para aprendizado e datasets médios, mas possui maior custo computacional em Big Data se comparada a operações vetorizadas.
- **Probabilidade de 1ª Ordem:** O operador de probabilidade condicional analisa apenas o vizinho imediato, não capturando dependências de longo prazo na sequência dos dados.



# Referências

- **HAN**, Jiawei; **KAMBER**, Micheline; **PEI**, Jian. *Mineração de Dados: conceitos e técnicas*. 3. ed. Rio de Janeiro: Elsevier, 2012.
- **BUSSAB**, Wilton de Oliveira; **MORETTIN**, Pedro Alberto. *Estatística Básica*. 9. ed. São Paulo: Saraiva, 2017.
- **TAN**, Pang-Ning; **STEINBACH**, Michael; **KUMAR**, Vipin. *Introdução ao Data Mining*. 1. ed. Rio de Janeiro: Ciência Moderna, 2009.
- **RAMALHO**, Luciano. *Python Fluente: Programação Clara, Concisa e Eficaz*. 1. ed. São Paulo: Novatec, 2015.
- **PYTHON SOFTWARE FOUNDATION**. *Python Language Reference, version 3.x*. Disponível em: <https://docs.python.org/3/>. Acesso em: 20 fev. 2026.